

Optimization Techniques for Real-Time Rendering of Procedurally Generated Scenes

Alex Antonio Soare

antonioalexsoare@gmail.com

Abstract

Procedural generation represents an important technique in the development of modern graphics applications, enabling the automatic creation of large and diverse virtual environments. However, the high geometric complexity of procedurally generated scenes can negatively affect the performance of interactive applications that require real-time rendering.

This project presents the development of a procedural generation system for 3D terrain and vegetation using the C++ programming language and the OpenGL graphics API. The system integrates procedural noise functions for terrain generation together with optimization techniques such as Level of Detail (LOD), Culling, and Instancing in order to reduce rendering costs.

The experimental results show that the application of these optimization techniques leads to significant performance improvements by reducing rendering time and the number of required graphical operations. The paper highlights the importance of integrating optimization mechanisms into modern graphics systems and demonstrates the effectiveness of procedural generation for building scalable and interactive 3D scenes.

1 Introduction

Modern graphics applications, including video games, interactive simulations, and virtual reality systems, require the real-time rendering of increasingly complex three-dimensional environments. The growing demand for visual realism leads to the processing of large amounts of geometric data and advanced lighting effects, which can significantly impact rendering performance and limit interactivity.

Procedural generation represents an important technique for the automatic creation of large-scale virtual environments. By utilizing mathematical

algorithms and procedural noise functions, terrains, vegetation, and other natural structures can be generated dynamically without explicit manual modeling. This approach reduces development time and memory requirements while enabling the creation of diverse, scalable, and theoretically infinite virtual worlds.

Despite these advantages, procedurally generated scenes often contain a very large number of geometric primitives and rendering operations, resulting in increased computational cost for both the CPU and GPU. Consequently, real-time applications require efficient rendering optimization techniques in order to maintain stable frame rates and interactive performance. In this context, optimization methods play a fundamental role in reducing overhead while preserving visual quality.

The goal of this project is to design and implement a procedural rendering system capable of generating and rendering terrain and vegetation in real time using the OpenGL graphics API and the C++ programming language. The proposed framework integrates procedural terrain generation based on fractal noise together with multiple rendering optimization techniques intended to improve scalability and reduce computational cost in large-scale scenes.

The main contribution of this paper consists of the development of a modular rendering pipeline that combines procedural terrain generation, procedural vegetation modeling, and real-time rendering optimizations within a unified framework. In addition, the paper provides an experimental evaluation of the implemented optimization methods by analyzing their impact on rendering performance using metrics such as frame rate, GPU frame time, CPU frame time, draw-call count, and memory usage.

The remainder of the paper is structured as follows. The first part presents the theoretical foundations of procedural generation, real-time rendering, and rendering optimization techniques. This is

followed by a description of the proposed system architecture and implementation details. Finally, the experimental results are presented and analyzed in order to evaluate the effectiveness of the optimization techniques and their impact on rendering performance.

2 Related Work and Theoretical Background

Procedural generation is widely used in computer graphics for the automatic creation of digital content, particularly in applications involving large-scale virtual environments. One of the fundamental components of procedural generation is the use of procedural noise functions, which enable the generation of continuous, non-repetitive, and visually natural structures through deterministic mathematical formulations.

Perlin Noise is one of the most widely used coherent noise functions in procedural graphics (Perlin, 1985). The algorithm produces smooth spatial variations by interpolating pseudo-random gradient vectors, making it suitable for terrain generation, procedural textures, cloud simulation, and other natural phenomena. Due to its continuity properties and visually organic appearance, Perlin Noise became a standard technique in procedural content generation.

A more computationally efficient alternative is Simplex Noise (Perlin, 2001), which was introduced to address several limitations of classic Perlin Noise. Unlike the original grid-based formulation, Simplex Noise utilizes simplex-based spatial decomposition, reducing directional artifacts and improving scalability in higher-dimensional spaces. This makes it particularly suitable for real-time procedural applications that require frequent noise evaluation on the GPU.

In order to generate more complex and realistic terrain formations, procedural systems frequently utilize fractal Brownian Motion (fBm), which combines multiple octaves of procedural noise evaluated at different frequencies and amplitudes. By combining noise layers at multiple spatial scales, fBm produces terrain structures that resemble real-world landscapes, including mountains, valleys, and small-scale surface irregularities.

The fBm function can be expressed as:

$$fBm(p) = \sum_{i=0}^{N-1} a_i \cdot noise(p \cdot f_i)$$

where a_i represents the amplitude of octave i , while f_i denotes the corresponding spatial frequency. Lower-frequency octaves contribute to the global terrain shape, whereas higher-frequency octaves introduce fine geometric detail.

Real-time rendering systems primarily rely on rasterization for image generation due to its computational efficiency and suitability for highly parallel GPU execution (Akenine-Moller and Haines, 2018). The OpenGL graphics pipeline consists of multiple processing stages, including vertex transformation, primitive assembly, rasterization, fragment processing, and framebuffer operations (Kessenich, 2016). Modern graphics hardware executes these stages in parallel, enabling interactive rendering performance even for geometrically complex scenes.

Lighting models play an important role in improving the visual realism of generated environments. In this project, scene illumination is implemented using the Blinn-Phong lighting model (Blinn, 1977), which extends the traditional Phong reflection model by introducing a halfway vector between the light direction and the viewing direction. This formulation reduces computational complexity while producing visually convincing specular highlights, making it suitable for real-time rendering applications.

In addition to direct illumination, the rendering system integrates projected vegetation shadows in order to improve depth perception and scene realism. Vegetation geometry is rendered in an additional rendering pass, where vertices are projected along the light direction onto the terrain surface. The projected shadow positions are computed directly in the vertex shader by evaluating terrain height values at the projected coordinates. Compared to more computationally expensive shadow-mapping approaches, this method provides a lightweight approximation suitable for large procedural scenes.

Large-scale procedural environments often contain millions of triangles and a very large number of rendering operations, making optimization techniques essential for maintaining interactive frame rates. Several rendering optimizations are therefore integrated into the proposed system (Akenine-Moller and Haines, 2018):

- **Level of Detail (LOD)** – dynamically reduces geometric complexity for distant terrain patches and vegetation objects;

- **Culling** – reduces unnecessary rendering operations through multiple visibility optimization techniques, including Frustum Culling, Occlusion Culling and Backface Culling;
- **Instancing** – minimizes CPU overhead by rendering multiple identical objects using shared geometry buffers and reduced draw calls.

Among terrain-specific optimization methods, GeoMipMapping (de Boer, 2000) is particularly important for procedural terrain rendering. The algorithm dynamically adjusts terrain mesh resolution according to camera distance, significantly reducing the number of processed triangles while preserving visual fidelity in regions close to the observer.

The integration of procedural generation techniques together with GPU-oriented rendering optimizations represents a fundamental requirement for modern real-time graphics systems. These methods enable the rendering of visually complex procedural environments while maintaining high performance, scalability, and interactive frame rates.

3 Methodology and Implementation

The proposed system is implemented in C++ using the OpenGL graphics API for rendering and ImGui for the graphical user interface (Kessenich, 2016). The application follows a modular architecture that separates procedural generation, rendering, lighting, and optimization components, facilitating independent testing, easier integration of rendering techniques, and improved scalability of the overall framework.

The system is designed around a dynamic terrain streaming approach, where terrain and vegetation data are generated and updated continuously according to the camera position. This architecture enables the simulation of large-scale or theoretically infinite virtual environments while maintaining controlled memory consumption and interactive rendering performance.

3.1 Terrain Generation

The terrain is divided into independent patches that are dynamically generated and updated according to the observer position. Only terrain regions located near the camera are actively maintained in memory and rendered, reducing unnecessary processing for distant or invisible regions.

Unlike traditional terrain systems based on pre-computed heightmaps, the proposed framework evaluates procedural noise functions directly on the GPU within the vertex shader stage. This approach eliminates the need for storing large terrain datasets in system memory while taking advantage of the massively parallel processing capabilities of modern graphics hardware.

Terrain elevation is generated procedurally using multiple octaves of fractal noise combined through fractal Brownian Motion (fBm) (Perlin, 1985, 2001). By blending noise layers evaluated at different frequencies and amplitudes, the generated terrain contains both large-scale geological formations and fine surface details. Lower-frequency octaves primarily define the global terrain shape, while higher-frequency octaves introduce local irregularities and surface variation.

The GPU-based terrain displacement pipeline also enables real-time modification of procedural parameters such as frequency, amplitude, persistence, and octave count without requiring geometry reconstruction on the CPU side. This significantly improves the flexibility and scalability of the procedural generation process.

To reduce geometric complexity, the terrain rendering system integrates the GeoMipMapping algorithm (de Boer, 2000), which dynamically adjusts terrain resolution according to camera distance. Each terrain patch can therefore be rendered using different levels of detail, significantly reducing the number of processed triangles for distant regions while preserving high visual fidelity near the observer. This adaptive resolution strategy substantially decreases GPU vertex-processing workload and improves rendering scalability in large procedural environments.

3.2 Procedural Vegetation Generation

Vegetation is generated procedurally using Lindenmayer-Systems (L-systems) (Prusinkiewicz and Lindenmayer, 1990), a rule-based formal grammar commonly used for modeling hierarchical branching structures such as trees and plants. Through iterative grammar expansion, the system generates recursive skeletal structures that approximate natural vegetation growth patterns.

The generated branch hierarchy is subsequently transformed into a three-dimensional mesh by constructing tubular geometry along branch segments. This process converts the abstract L-system repre-

sensation into renderable polygonal geometry suitable for real-time rendering.

In order to improve visual realism and reduce repetitive patterns, several procedural parameters are randomized during generation, including branch length, branching angle, scaling factors, curvature variation, and branch survival probability. These stochastic variations produce unique vegetation instances while preserving the global structural characteristics defined by the procedural grammar.

Vegetation placement is also performed procedurally for each terrain patch. The distribution process depends on terrain elevation, slope, and procedural noise values, enabling the generation of diverse environments. This procedural distribution contributes to the visual diversity and natural appearance of the generated scenes.

3.3 Optimization Techniques

Large-scale procedural scenes contain a substantial amount of geometric data and rendering operations, making optimization techniques essential for maintaining interactive frame rates (Akenine-Moller and Haines, 2018). Multiple GPU-oriented optimization methods were therefore integrated into the rendering pipeline.

- **Terrain LOD** – reduces geometric complexity for distant terrain patches using GeoMipMapping-based adaptive mesh resolution;
- **Vegetation LOD** – simplifies procedural vegetation models by reducing branch count and mesh complexity for distant objects;
- **Instancing** – minimizes CPU overhead and draw-call count by rendering multiple vegetation instances using shared geometry buffers;
- **Frustum Culling** – eliminates terrain patches and vegetation objects located outside the camera view volume before submission to the GPU;
- **Backface Culling** – avoids rasterization of polygons oriented away from the camera, reducing fragment-processing workload.

The combination of these optimization strategies significantly reduces rendering overhead on both the CPU and GPU. In particular, instanced

rendering minimizes communication overhead between CPU and GPU, while LOD techniques reduce vertex-processing costs by dynamically adapting geometric complexity according to visual importance.

The implemented optimization pipeline allows the system to maintain high rendering performance even in scenes containing dense procedural vegetation and large terrain areas.

3.4 Lighting and Shadows

Scene illumination is implemented using the Blinn-Phong lighting model in the fragment shader stage (Blinn, 1977). The model computes ambient, diffuse, and specular lighting components while remaining computationally efficient for real-time rendering applications.

Compared to the traditional Phong reflection model, the Blinn-Phong formulation utilizes a halfway vector between the light direction and the viewing direction, reducing computational complexity while preserving visually convincing specular highlights.

To improve visual realism and depth perception, the rendering system also integrates projected vegetation shadows. After the terrain rendering stage, vegetation shadows are rendered in a dedicated additional pass directly into the scene. During this process, vegetation vertices are projected onto the terrain surface along the light direction.

The projection calculations are performed directly in the vertex shader by tracing the projected position along the light vector and evaluating terrain height values at the projected coordinates. The resulting projected geometry is subsequently rendered using the standard camera view and projection matrices.

Compared to more computationally expensive shadow-mapping approaches, the proposed projected-shadow method provides a lightweight approximation suitable for large procedural scenes while maintaining acceptable visual quality and interactive frame rates.

The integration of procedural terrain generation, vegetation synthesis, adaptive rendering optimizations, and real-time lighting enables the proposed framework to render visually complex environments efficiently while preserving scalability and rendering performance.

4 Results and Metrics

The performance of the proposed system was evaluated using several metrics, including frames per second (FPS), CPU frame time, GPU frame time, memory usage, and draw-call count. In order to analyze the rendering pipeline accurately, a custom profiling system was integrated directly into the application.

The profiler monitors both CPU-side and GPU-side execution costs in real time. CPU timings were obtained by measuring the duration of each rendered frame, while GPU timings were measured using OpenGL timer queries (Kessenich, 2016). The profiler also supports named GPU profiling sections, allowing independent analysis of terrain rendering, vegetation rendering, shadow rendering, and optimization stages. Additionally, profiling data can be exported in CSV format and processed externally using Python scripts for statistical analysis and graph generation.

Experimental evaluation demonstrated that rendering optimization techniques have a major impact on performance in large procedural scenes. In particular, the Level of Detail (LOD) system proved highly effective for reducing geometric complexity and GPU workload.

In the initial high-detail configuration, the application rendered complex terrain and dense procedural vegetation without aggressive optimization. Under these conditions, GPU frame times frequently exceeded 25 ms per frame, resulting in performance close to 40 FPS and indicating a GPU-bound rendering pipeline.

After enabling terrain LOD optimization using GeoMipMapping (de Boer, 2000), the number of rendered terrain triangles was significantly reduced for distant patches. This optimization alone increased rendering performance beyond 100 FPS in most test scenarios.

The most significant improvement was achieved through vegetation simplification. Procedural vegetation represented the dominant source of geometric complexity due to the large number of generated branches and mesh segments. After reducing vegetation detail levels and simplifying branch geometry, GPU frame times dropped below 5 ms per frame, while frame rates consistently exceeded 200 FPS. Statistical analysis also indicated improved frame stability, with high-percentile frame times (p95 and p99) decreasing substantially.

Instanced rendering also produced major perfor-

mance improvements, particularly in scenes containing dense vegetation. Multiple experiments were conducted using vegetation densities ranging from 10 to 40 instances per terrain patch.

Without instancing, rendering performance degraded rapidly as vegetation density increased. In the most demanding configuration, the number of draw calls exceeded 9000, producing significant CPU overhead and causing frame rates to fall below 100 FPS. After enabling instancing, vegetation objects were rendered using shared geometry buffers and a reduced number of rendering commands. As a result, frame rates increased substantially in all tested scenarios, reaching nearly 300 FPS for low-density scenes and remaining above 140 FPS even in the most demanding configurations.

Frustum Culling further improved performance by eliminating terrain patches and vegetation objects located outside the camera view volume (Akenine-Moller and Haines, 2018). This reduced unnecessary GPU processing and minimized rendering overhead for invisible geometry.

The lighting system and projected vegetation shadows introduced additional rendering costs but significantly improved visual realism. The projected shadows enhanced depth perception and created more natural interactions between vegetation and terrain while preserving interactive frame rates.

Overall, the experimental results demonstrate that combining procedural generation with GPU-oriented optimization techniques enables the rendering of visually complex procedural environments while maintaining high performance, scalability, and stable real-time frame rates.

5 Future Work

Future development directions for the system include:

- implementation of Physically Based Rendering (PBR) techniques for more realistic materials and lighting;
- integration of advanced occlusion culling algorithms;
- procedural generation of complex ecosystems and biome systems;
- support for hardware-accelerated ray tracing;
- GPU compute shader implementations for procedural generation and vegetation simulation.

6 Conclusions

The paper presented the development of a procedural terrain and vegetation generation system optimized for real-time rendering. The proposed framework combines procedural noise-based terrain generation with rendering optimization techniques.

The application was designed around a modular architecture that separates procedural generation, rendering, lighting, and optimization components. Terrain generation is performed dynamically using fractal noise evaluated directly on the GPU, enabling the creation of theoretically infinite virtual environments while maintaining efficient memory usage. Procedural vegetation generation based on L-systems allows the creation of natural branching structures with controllable complexity and visual variation.

The system also integrates a real-time lighting pipeline based on the Blinn-Phong illumination model together with projected vegetation shadows in order to improve visual realism and depth perception. In addition, the configurable graphical interface developed using ImGui enables real-time modification of procedural parameters and rendering settings, facilitating experimental evaluation and performance analysis.

The experimental results demonstrated that rendering optimization techniques significantly improve performance in large-scale procedural scenes. The LOD system effectively reduced geometric complexity for distant terrain and vegetation, while instanced rendering substantially decreased CPU overhead and draw-call count. Culling further reduced unnecessary rendering operations by eliminating invisible geometry before submission to the GPU.

Performance profiling showed that these optimizations enabled the application to maintain stable real-time frame rates even in highly complex scenes containing dense procedural vegetation and large terrain areas. The integrated profiling framework also provided detailed CPU and GPU performance analysis, allowing accurate evaluation of the impact of each optimization method.

Overall, the project highlights the importance of oriented optimization techniques in modern computer graphics applications and demonstrates the potential of procedural generation as a scalable solution for the development of large, complex, and interactive virtual environments. The obtained results show that combining procedural content

generation with adaptive rendering optimizations enables efficient real-time rendering while preserving visual quality and scalability.

7 Limitations

Although the proposed system achieves high rendering performance, some limitations remain.

One important limitation is related to terrain generation directly inside the GPU vertex shader. Since the procedural noise is evaluated exclusively on the GPU, exact terrain height values are not directly accessible on the CPU side. While this approach improves scalability and reduces memory usage, it complicates the implementation of additional CPU-side systems.

The current implementation of Frustum Culling also introduces certain inefficiencies. Terrain visibility testing is based on conservative Axis-Aligned Bounding Boxes (AABB), which are intentionally oversized because the exact terrain deformation is computed dynamically on the GPU. As a result, some terrain patches may still be rendered even when only a very small visible region intersects the camera frustum. Although this approach avoids visual artifacts such as premature terrain disappearance, it reduces culling precision and introduces additional rendering overhead.

Another limitation is related to the lighting system. The projected vegetation shadow technique provides a computationally efficient approximation suitable for real-time rendering, but it does not achieve the visual accuracy of more advanced shadow-mapping or ray-tracing approaches.

References

- Tomas Akenine-Moller and Eric Haines. 2018. *Real-Time Rendering*. CRC Press.
- James F. Blinn. 1977. Models of light reflection for computer synthesized pictures. *ACM SIGGRAPH Computer Graphics*.
- Willem de Boer. 2000. Geomipmapping: A real-time rendering algorithm for terrain visualization.
- John Kessenich. 2016. *OpenGL Programming Guide*. Addison-Wesley.
- Ken Perlin. 1985. An image synthesizer. *ACM SIGGRAPH*.
- Ken Perlin. 2001. Simplex noise demystified.
- Przemyslaw Prusinkiewicz and Aristid Lindenmayer. 1990. *The Algorithmic Beauty of Plants*. Springer-Verlag.